

RTOS_Echtzeitsystem

Generated by Doxygen 1.17.0

Chapter 1

Topic Index

1.1 Topics

Here is a list of all topics with brief descriptions:

CMSIS	??
Stm32l4xx_system	??
STM32L4xx_System_Private_Includes	??
STM32L4xx_System_Private_TypesDefinitions	??
STM32L4xx_System_Private_Defines	??
STM32L4xx_System_Private_Macros	??
STM32L4xx_System_Private_Variables	??
STM32L4xx_System_Private_FunctionPrototypes	??
STM32L4xx_System_Private_Functions	??

Chapter 2

File Index

2.1 File List

Here is a list of all files with brief descriptions:

main.c	: Main program body	??
MessageQueue.c	Implementierung einer Message Queue für die Inter-Task-Kommunikation	??
Mutex_and_Semaphoren.c	Implementierung von Mutexen und Semaphoren für die Task-Synchronisation	??
os_kernel.c	Kernfunktionen des Real-Time Operating Systems (RTOS)	??
stm32l4xx_hal_msp.c	This file provides code for the MSP Initialization and de-Initialization codes	??
stm32l4xx_it.c	Interrupt Service Routines	??
syscalls.c	STM32CubeIDE Minimal System calls file	??
systemem.c	STM32CubeIDE System Memory calls file	??
system_stm32l4xx.c	CMSIS Cortex-M4 Device Peripheral Access Layer System Source File	??

Chapter 3

Topic Documentation

3.1 CMSIS

Topics

- [Stm32l4xx_system](#)

3.1.1 Detailed Description

3.1.2 Stm32l4xx_system

Topics

- [STM32L4xx_System_Private_Includes](#)
- [STM32L4xx_System_Private_TypesDefinitions](#)
- [STM32L4xx_System_Private_Defines](#)
- [STM32L4xx_System_Private_Macros](#)
- [STM32L4xx_System_Private_Variables](#)
- [STM32L4xx_System_Private_FunctionPrototypes](#)
- [STM32L4xx_System_Private_Functions](#)

3.1.2.1 Detailed Description

3.1.2.2 STM32L4xx_System_Private_Includes

3.1.2.3 STM32L4xx_System_Private_TypesDefinitions

3.1.2.4 STM32L4xx_System_Private_Defines

Macros

- `#define HSE\_VALUE 8000000U`
- `#define MSI\_VALUE 4000000U`
- `#define HSI\_VALUE 16000000U`

3.1.2.4.1 Detailed Description

3.1.2.4.2 Macro Definition Documentation

3.1.2.4.2.1 HSE_VALUE

```
#define HSE_VALUE 8000000U
```

Value of the External oscillator in Hz

3.1.2.4.2.2 HSI_VALUE

```
#define HSI_VALUE 16000000U
```

Value of the Internal oscillator in Hz

3.1.2.4.2.3 MSI_VALUE

```
#define MSI_VALUE 4000000U
```

Value of the Internal oscillator in Hz

3.1.2.5 STM32L4xx_System_Private_Macros

3.1.2.6 STM32L4xx_System_Private_Variables

Variables

- uint32_t [SystemCoreClock](#) = 4000000U
- const uint8_t [AHBPrescTable](#) [16] = {0U, 0U, 0U, 0U, 0U, 0U, 0U, 0U, 1U, 2U, 3U, 4U, 6U, 7U, 8U, 9U}
- const uint8_t [APBPrescTable](#) [8] = {0U, 0U, 0U, 0U, 1U, 2U, 3U, 4U}
- const uint32_t [MSIRangeTable](#) [12]

3.1.2.6.1 Detailed Description

3.1.2.6.2 Variable Documentation

3.1.2.6.2.1 AHBPrescTable

```
const uint8_t AHBPrescTable[16] = {0U, 0U, 0U, 0U, 0U, 0U, 0U, 0U, 0U, 1U, 2U, 3U, 4U, 6U, 7U, 8U, 9U}
```

3.1.2.6.2.2 APBPrescTable

```
const uint8_t APBPrescTable[8] = {0U, 0U, 0U, 0U, 1U, 2U, 3U, 4U}
```


3.1.2.6.2.3 MSIRangeTable

```
const uint32_t MSIRangeTable[12]
```

Initial value:

```
= {1000000U, 2000000U, 4000000U, 8000000U, 10000000U, 20000000U,  
   40000000U, 80000000U, 160000000U, 240000000U, 320000000U, 480000000U}
```

3.1.2.6.2.4 SystemCoreClock

```
uint32_t SystemCoreClock = 4000000U
```

3.1.2.7 STM32L4xx_System_Private_FunctionPrototypes

3.1.2.8 STM32L4xx_System_Private_Functions

Functions

- void [SystemInit](#) (void)
Setup the microcontroller system.
- void [SystemCoreClockUpdate](#) (void)
Update SystemCoreClock variable according to Clock Register Values. The SystemCoreClock variable contains the core clock (HCLK), it can be used by the user application to setup the SysTick timer or configure other parameters.

3.1.2.8.1 Detailed Description

3.1.2.8.2 Function Documentation

3.1.2.8.2.1 SystemCoreClockUpdate()

```
void SystemCoreClockUpdate (  
    void )
```

Update SystemCoreClock variable according to Clock Register Values. The SystemCoreClock variable contains the core clock (HCLK), it can be used by the user application to setup the SysTick timer or configure other parameters.

Note

Each time the core clock (HCLK) changes, this function must be called to update SystemCoreClock variable value. Otherwise, any configuration based on this variable will be incorrect.

- The system frequency computed by this function is not the real frequency in the chip. It is calculated based on the predefined constant and the selected clock source:

- If SYSCLK source is MSI, SystemCoreClock will contain the [MSI_VALUE\(*\)](#)
- If SYSCLK source is HSI, SystemCoreClock will contain the [HSI_VALUE\(**\)](#)
- If SYSCLK source is HSE, SystemCoreClock will contain the [HSE_VALUE\(***\)](#)
- If SYSCLK source is PLL, SystemCoreClock will contain the [HSE_VALUE\(***\)](#) or [HSI_VALUE\(*\)](#) or [MSI_VALUE\(*\)](#) multiplied/divided by the PLL factors.

(*) MSI_VALUE is a constant defined in stm32l4xx_hal.h file (default value 4 MHz) but the real value may vary depending on the variations in voltage and temperature.

(**) HSI_VALUE is a constant defined in stm32l4xx_hal.h file (default value 16 MHz) but the real value may vary depending on the variations in voltage and temperature.

(***) HSE_VALUE is a constant defined in stm32l4xx_hal.h file (default value 8 MHz), user has to ensure that HSE_VALUE is same as the real frequency of the crystal used. Otherwise, this function may have wrong result.

- The result of this function could be not correct when using fractional value for HSE crystal.

Return values

None	
------	--

3.1.2.8.2.2 SystemInit()

```
void SystemInit (  
    void )
```

Setup the microcontroller system.

Return values

None	
------	--

Chapter 4

File Documentation

4.1 main.c File Reference

: Main program body

```
#include "main.h"
#include "stm32l4xx.h"
#include "os_kernel.h"
#include "tcb.h"
#include "Mutex_and_Semaphoren.h"
#include "MessageQueue.h"
#include "os_sync.h"
```

Functions

- void [SystemClock_Config](#) (void)
System Clock Configuration.
- void [Task_BlinkGreen](#) ()
- void [Task_BlinkRed](#) ()
- void [Task_Idle](#) (void)
- int [main](#) (void)
The application entry point.
- void [Error_Handler](#) (void)
This function is executed in case of error occurrence.

Variables

- uint32_t [MAX_TASKS](#) = 3
Externe Variable, definiert die absolute maximale Task-Anzahl.
- DFSDM_Channel_HandleTypeDef [hdfsdm1_channel1](#)
- I2C_HandleTypeDef [hi2c2](#)
- QSPI_HandleTypeDef [hqspi](#)
- SPI_HandleTypeDef [hsapi3](#)
- UART_HandleTypeDef [huart1](#)
- UART_HandleTypeDef [huart3](#)
- PCD_HandleTypeDef [hpcd_USB_OTG_FS](#)

4.1.1 Detailed Description

: Main program body

Attention

Copyright (c) 2026 STMicroelectronics. All rights reserved.

This software is licensed under terms that can be found in the LICENSE file in the root directory of this software component. If no LICENSE file comes with this software, it is provided AS-IS.

4.1.2 Function Documentation

4.1.2.1 Error_Handler()

```
void Error_Handler (  
    void )
```

This function is executed in case of error occurrence.

Return values

<i>None</i>	
-------------	--

4.1.2.2 main()

```
int main (  
    void )
```

The application entry point.

Return values

<i>int</i>	
------------	--

4.1.2.3 SystemClock_Config()

```
void SystemClock_Config (
    void )
```

System Clock Configuration.

Return values

None	
------	--

Configure the main internal regulator output voltage

Configure LSE Drive Capability

Initializes the RCC Oscillators according to the specified parameters in the RCC_OscInitTypeDef structure.

Initializes the CPU, AHB and APB buses clocks

Enable MSI Auto calibration

4.1.2.4 Task_BlinkGreen()

```
void Task_BlinkGreen ()
```

4.1.2.5 Task_BlinkRed()

```
void Task_BlinkRed ()
```

4.1.2.6 Task_Idle()

```
void Task_Idle (
    void )
```

4.1.3 Variable Documentation

4.1.3.1 hdfsdm1_channel1

```
DFSDM_Channel_HandleTypeDef hdfsdm1_channel1
```

4.1.3.2 hi2c2

```
I2C_HandleTypeDef hi2c2
```

4.1.3.3 hpcd_USB_OTG_FS

```
PCD_HandleTypeDef hpcd_USB_OTG_FS
```

4.1.3.4 hqspi

```
QSPI_HandleTypeDef hqspi
```

4.1.3.5 hspi3

```
SPI_HandleTypeDef hspi3
```

4.1.3.6 huart1

```
UART_HandleTypeDef huart1
```

4.1.3.7 huart3

```
UART_HandleTypeDef huart3
```

4.1.3.8 MAX_TASKS

```
uint32_t MAX_TASKS = 3
```

Externe Variable, definiert die absolute maximale Task-Anzahl.

4.2 MessageQueue.c File Reference

Implementierung einer Message Queue für die Inter-Task-Kommunikation.

```
#include "MessageQueue.h"
```

Macros

- `#define Max_Task 3`
Maximale Anzahl an Tasks, die vom OS verwaltet werden.

Functions

- void [OS_MsgQ_Init](#) (OS_MessageQueue_t *pQ, uint32_t *pBufferArray, uint32_t u32Capacity)
Initialisiert eine Message Queue.
- uint8_t [OS_MsgQ_IsEmpty](#) (OS_MessageQueue_t *pQ)
Prüft, ob die Message Queue leer ist.
- uint8_t [OS_MsgQ_IsFull](#) (OS_MessageQueue_t *pQ)
Prüft, ob die Message Queue voll ist.
- uint8_t [OS_MsgQ_SendNonBlocking](#) (OS_MessageQueue_t *pQ, uint32_t u32Msg)
Sendet eine Nachricht in die Queue (Non-Blocking).
- void [OS_MsgQ_SendBlocking](#) (OS_MessageQueue_t *pQ, uint32_t u32Msg)
Sendet eine Nachricht in die Queue (Blocking).
- uint8_t [OS_MsgQ_SendTimeout](#) (OS_MessageQueue_t *pQ, uint32_t u32Msg, uint32_t u32TimeoutTicks)
Sendet eine Nachricht in die Queue mit Timeout.
- uint8_t [OS_MsgQ_ReadNonBlocking](#) (OS_MessageQueue_t *pQ, uint32_t *pDest)
Liest eine Nachricht aus der Queue (Non-Blocking).
- void [OS_MsgQ_ReadBlocking](#) (OS_MessageQueue_t *pQ, uint32_t *pDest)
Liest eine Nachricht aus der Queue (Blocking).
- uint8_t [OS_MsgQ_ReadTimeout](#) (OS_MessageQueue_t *pQ, uint32_t *pDest, uint32_t u32TimeoutTicks)
Liest eine Nachricht aus der Queue mit Timeout.

4.2.1 Detailed Description

Implementierung einer Message Queue für die Inter-Task-Kommunikation.

Diese Datei enthält die Implementierung einer threadsicheren Message Queue basierend auf einem Ringpuffer (Circular Buffer). Sie bietet Mechanismen für Non-Blocking, Blocking und Timeout-basiertes Senden und Empfangen von Nachrichten. Die Synchronisation erfolgt über globale Interrupt-Sperren und das Auslösen eines PendSV-Interrupts für den Context Switch.

Author

ezech

Date

4. Juni 2026

4.2.2 Macro Definition Documentation

4.2.2.1 Max_Task

```
#define Max_Task 3
```

Maximale Anzahl an Tasks, die vom OS verwaltet werden.

4.2.3 Function Documentation

4.2.3.1 OS_MsgQ_Init()

```
void OS_MsgQ_Init (
    OS_MessageQueue_t * pQ,
    uint32_t * pBufferArray,
    uint32_t u32Capacity)
```

Initialisiert eine Message Queue.

Setzt alle internen Zeiger und Zähler der Queue auf 0 und verknüpft sie mit dem bereitgestellten Speicherbereich.

Parameters

<i>pQ</i>	Zeiger auf die zu initialisierende OS_MessageQueue_t Struktur.
<i>pBufferArray</i>	Zeiger auf das Array, das als Ringpuffer dienen soll.
<i>u32Capacity</i>	Maximale Anzahl an Elementen, die die Queue aufnehmen kann.

4.2.3.2 OS_MsgQ_IsEmpty()

```
uint8_t OS_MsgQ_IsEmpty (
    OS_MessageQueue_t * pQ)
```

Prüft, ob die Message Queue leer ist.

Parameters

<i>pQ</i>	Zeiger auf die Message Queue.
-----------	-------------------------------

Returns

1 wenn die Queue leer ist, sonst 0.

4.2.3.3 OS_MsgQ_IsFull()

```
uint8_t OS_MsgQ_IsFull (
    OS_MessageQueue_t * pQ)
```

Prüft, ob die Message Queue voll ist.

Parameters

<i>pQ</i>	Zeiger auf die Message Queue.
-----------	-------------------------------

Returns

1 wenn die Queue voll ist, sonst 0.

4.2.3.4 OS_MsgQ_ReadBlocking()

```
void OS_MsgQ_ReadBlocking (
    OS_MessageQueue_t * pQ,
    uint32_t * pDest)
```

Liest eine Nachricht aus der Queue (Blocking).

Wenn die Queue leer ist, blockiert der Task in der Bitmaske u32WaitingReaders, bis ein Writer-Task eine neue Nachricht sendet.

Parameters

<i>pQ</i>	Zeiger auf die Message Queue.
<i>pDest</i>	Zeiger auf die Zielvariable für die gelesene Nachricht.

4.2.3.5 OS_MsgQ_ReadNonBlocking()

```
uint8_t OS_MsgQ_ReadNonBlocking (
    OS_MessageQueue_t * pQ,
    uint32_t * pDest)
```

Liest eine Nachricht aus der Queue (Non-Blocking).

Holt die älteste Nachricht aus dem Ringpuffer, falls vorhanden. Weckt bei erfolgreichem Lesen prioritätsbasiert einen wartenden Writer-Task auf.

Parameters

<i>pQ</i>	Zeiger auf die Message Queue.
<i>pDest</i>	Zeiger auf die Variable, in die die Nachricht kopiert werden soll.

Returns

1 bei Erfolg, 0 wenn die Queue leer war.

4.2.3.6 OS_MsgQ_ReadTimeout()

```
uint8_t OS_MsgQ_ReadTimeout (
    OS_MessageQueue_t * pQ,
    uint32_t * pDest,
    uint32_t u32TimeoutTicks)
```

Liest eine Nachricht aus der Queue mit Timeout.

Versucht zu lesen. Ist die Queue leer, blockiert der Task maximal für die angegebene Zeit. Erfolgt in dieser Zeit kein Schreibvorgang, kehrt die Funktion mit einem Timeout-Fehler zurück.

Parameters

<i>pQ</i>	Zeiger auf die Message Queue.
<i>pDest</i>	Zeiger auf die Zielvariable für die gelesene Nachricht.
<i>u32TimeoutTicks</i>	Maximale Wartezeit in OS-Ticks.

Returns

1 bei erfolgreichem Lesen, 0 bei einem Timeout.

4.2.3.7 OS_MsgQ_SendBlocking()

```
void OS_MsgQ_SendBlocking (
    OS_MessageQueue_t * pQ,
    uint32_t u32Msg)
```

Sendet eine Nachricht in die Queue (Blocking).

Wenn die Queue voll ist, wird der aufrufende Task blockiert und in der Bitmaske u32WaitingWriters registriert. Er wird erst wieder fortgesetzt, wenn ein anderer Task eine Nachricht ausliest und Platz schafft.

Parameters

<i>pQ</i>	Zeiger auf die Message Queue.
<i>u32Msg</i>	Die zu sendende Nachricht.

4.2.3.8 OS_MsgQ_SendNonBlocking()

```
uint8_t OS_MsgQ_SendNonBlocking (
    OS_MessageQueue_t * pQ,
    uint32_t u32Msg)
```

Sendet eine Nachricht in die Queue (Non-Blocking).

- Versucht, eine Nachricht in die Queue zu schreiben. Wenn die Queue voll ist, kehrt die Funktion sofort zurück, ohne den Task zu blockieren. Weckt bei Erfolg prioritätsbasiert einen wartenden Reader-Task auf.

Parameters

<i>pQ</i>	Zeiger auf die Message Queue.
<i>u32Msg</i>	Die zu sendende Nachricht (32-Bit Wert).

Returns

1 bei erfolgreichem Senden, 0 wenn die Queue voll war.

4.2.3.9 OS_MsgQ_SendTimeout()

```
uint8_t OS_MsgQ_SendTimeout (
    OS_MessageQueue_t * pQ,
    uint32_t u32Msg,
    uint32_t u32TimeoutTicks)
```

Sendet eine Nachricht in die Queue mit Timeout.

Versucht zu schreiben. Ist die Queue voll, blockiert der Task maximal für die angegebene Anzahl an Ticks. Wird er durch den SysTick-Handler vorzeitig geweckt, schlägt das Senden fehl.

Parameters

<i>pQ</i>	Zeiger auf die Message Queue.
<i>u32Msg</i>	Die zu sendende Nachricht.
<i>u32TimeoutTicks</i>	Maximale Wartezeit in OS-Ticks.

Returns

1 bei Erfolg, 0 bei einem Timeout.

4.3 Mutex_and_Semaphoren.c File Reference

Implementierung von Mutexen und Semaphoren für die Task-Synchronisation.

```
#include "tcb.h"
#include "os_sync.h"
#include "Mutex_and_Semaphoren.h"
```

Macros

- #define [Max_Task](#) 3

Functions

- void [OS_Mutex_Init](#) (OS_Mutex_t *pMutex)
Initialisiert ein Mutex-Objekt.
- uint8_t [OS_Mutex_AcquireNonBlocking](#) (OS_Mutex_t *pMutex)
Fordert einen Mutex an (Non-Blocking).
- void [OS_Mutex_AcquireBlocking](#) (OS_Mutex_t *pMutex)
Fordert einen Mutex an (Blocking).
- uint8_t [OS_Mutex_AcquireTimeout](#) (OS_Mutex_t *pMutex, uint32_t u32TimeoutTicks)
Fordert einen Mutex mit Timeout an.
- void [OS_Mutex_Release](#) (OS_Mutex_t *pMutex)
Gibt einen Mutex frei.
- void [OS_Semaphore_init](#) (OS_Semaphore_t *pSem, uint32_t u32InitCount, uint32_t u32MaxCount)
Initialisiert eine Semaphore.

- `uint8_t OS_Sem_AcquireNonBlocking` (`OS_Semaphore_t *pSem`)
Fordert ein Semaphore-Token an (Non-Blocking).
- `void OS_Sem_AcquireBlocking` (`OS_Semaphore_t *pSem`)
Fordert ein Semaphore-Token an (Blocking).
- `uint8_t OS_Sem_AcquireTimeout` (`OS_Semaphore_t *pSem`, `uint32_t u32TimeoutTicks`)
Fordert ein Semaphore-Token an mit Timeout.
- `void OS_Sem_Release` (`OS_Semaphore_t *pSem`)
Gibt ein Semaphore-Token frei oder signalisiert ein Event.

4.3.1 Detailed Description

Implementierung von Mutexen und Semaphoren für die Task-Synchronisation.

Diese Datei enthält die threadsicheren Implementierungen für Mutexe und zählende Semaphoren. Sie ermöglichen das sichere Teilen von Ressourcen sowie die Signalisierung von Events zwischen verschiedenen Tasks. Die Funktionen unterstützen Non-Blocking, Blocking und Timeout-basierte Zugriffe.

Author

Ezechiel Tonkeme

Date

30. Mai 2026

4.3.2 Macro Definition Documentation

4.3.2.1 Max_Task

```
#define Max_Task 3
```

4.3.3 Function Documentation

4.3.3.1 OS_Mutex_AcquireBlocking()

```
void OS_Mutex_AcquireBlocking (
    OS_Mutex_t * pMutex)
```

Fordert einen Mutex an (Blocking).

Ist der Mutex belegt, wird der aktuelle Task in den Zustand `TaskState_Blocked` versetzt und in der Wartemaske des Mutex registriert. Anschließend wird ein Context Switch (`PendSV`) erzwungen.

Parameters

<i>pMutex</i>	Zeiger auf das angeforderte Mutex-Objekt.
---------------	---

4.3.3.2 OS_Mutex_AcquireNonBlocking()

```
uint8_t OS_Mutex_AcquireNonBlocking (  
    OS_Mutex_t * pMutex)
```

Fordert einen Mutex an (Non-Blocking).

Versucht, den Mutex zu sperren. Ist dieser bereits belegt, kehrt die Funktion sofort zurück, ohne den aufrufenden Task zu blockieren.

Parameters

<i>pMutex</i>	Zeiger auf das angeforderte Mutex-Objekt.
---------------	---

Returns

1 bei erfolgreichem Acquire, 0 wenn der Mutex bereits belegt ist.

4.3.3.3 OS_Mutex_AcquireTimeout()

```
uint8_t OS_Mutex_AcquireTimeout (  
    OS_Mutex_t * pMutex,  
    uint32_t u32TimeoutTicks)
```

Fordert einen Mutex mit Timeout an.

Blockiert den Task maximal für die angegebene Zeitspanne (Ticks). Wird der Task durch einen anderen Task geweckt (Release), erhält er den Mutex. Läuft der Timer ab (SysTick), schlägt das Acquire fehl.

Parameters

<i>pMutex</i>	Zeiger auf das angeforderte Mutex-Objekt.
<i>u32TimeoutTicks</i>	Maximale Wartezeit in OS-Ticks.

Returns

1 bei erfolgreichem Acquire, 0 bei einem Timeout.

4.3.3.4 OS_Mutex_Init()

```
void OS_Mutex_Init (  
    OS_Mutex_t * pMutex)
```

Initialisiert ein Mutex-Objekt.

Setzt den Lock-Status auf frei und weist eine ungültige Owner-ID (255) zu, um zu signalisieren, dass der Mutex aktuell keinem Task gehört.

Parameters

<i>pMutex</i>	Zeiger auf das zu initialisierende OS_Mutex_t Objekt.
---------------	---

4.3.3.5 OS_Mutex_Release()

```
void OS_Mutex_Release (
    OS_Mutex_t * pMutex)
```

Gibt einen Mutex frei.

Überprüft zunächst, ob der aufrufende Task tatsächlich der Besitzer ist. Sind andere Tasks in der Warteliste, wird der Mutex direkt an den wartenden Task mit der entsprechenden Priorität (bzw. ID) übergeben und ein Context Switch angefordert.

Parameters

<i>pMutex</i>	Zeiger auf das freizugebende Mutex-Objekt.
---------------	--

4.3.3.6 OS_Sem_AcquireBlocking()

```
void OS_Sem_AcquireBlocking (
    OS_Semaphore_t * pSem)
```

Fordert ein Semaphore-Token an (Blocking).

Wenn keine Token verfügbar sind (Count == 0), wird der Task blockiert und ein Context Switch ausgelöst, um CPU-Zeit zu sparen.

Parameters

<i>pSem</i>	Zeiger auf das angeforderte Semaphore-Objekt.
-------------	---

4.3.3.7 OS_Sem_AcquireNonBlocking()

```
uint8_t OS_Sem_AcquireNonBlocking (
    OS_Semaphore_t * pSem)
```

Fordert ein Semaphore-Token an (Non-Blocking).

Parameters

<i>pSem</i>	Zeiger auf das angeforderte Semaphore-Objekt.
-------------	---

Returns

1 wenn ein Token erfolgreich abgezogen wurde, 0 wenn kein Token verfügbar war.

4.3.3.8 OS_Sem_AcquireTimeout()

```
uint8_t OS_Sem_AcquireTimeout (
    OS_Semaphore_t * pSem,
    uint32_t u32TimeoutTicks)
```

Fordert ein Semaphore-Token an mit Timeout.

Versetzt den Task in den Blocked-Status, falls kein Token vorhanden ist. Der Task wacht entweder durch ein Timeout (ausgelöst vom SysTick) oder durch die Freigabe eines Tokens (Release) wieder auf.

Parameters

<i>pSem</i>	Zeiger auf das angeforderte Semaphore-Objekt.
<i>u32TimeoutTicks</i>	Maximale Wartezeit in OS-Ticks.

Returns

1 wenn das Token innerhalb der Zeit erhalten wurde, 0 bei Timeout.

4.3.3.9 OS_Sem_Release()

```
void OS_Sem_Release (
    OS_Semaphore_t * pSem)
```

Gibt ein Semaphore-Token frei oder signalisiert ein Event.

Wenn sich Tasks in der Warteschlange befinden, wird der Zähler (Count) nicht erhöht. Stattdessen wird direkt ein wartender Task geweckt (TaskState_Ready) und ein Context Switch angefordert. Andernfalls wird der Token-Zähler inkrementiert.

Parameters

<i>pSem</i>	Zeiger auf das freizugebende Semaphore-Objekt.
-------------	--

4.3.3.10 OS_Semaphore_init()

```
void OS_Semaphore_init (
    OS_Semaphore_t * pSem,
    uint32_t u32InitCount,
    uint32_t u32MaxCount)
```

Initialisiert eine Semaphore.

Parameters

<i>pSem</i>	Zeiger auf das zu initialisierende OS_Semaphore_t Objekt.
<i>u32InitCount</i>	Anfängliche Anzahl der verfügbaren Token/Ressourcen.

<code>u32MaxCount</code>	Maximale Anzahl an Token, die diese Semaphore aufnehmen kann.
--------------------------	---

4.4 os_kernel.c File Reference

Kernfunktionen des Real-Time Operating Systems (RTOS).

```
#include "stm32l4xx.h"
#include "os_kernel.h"
#include "tcb.h"
```

Macros

- `#define Max_Task 3`
Maximale Anzahl an Tasks, die vom OS verwaltet werden (inkl. Idle-Task).

Typedefs

- `typedef void(* OS_TaskFunction_t) (void)`

Functions

- `void OS_StartFirstTask (void)`
- `uint8_t OS_TaskCreate (OS_TaskFunction_t pTaskCode, uint8_t u8Prio, uint8_t u8TaskId)`
Erstellt einen neuen Task und initialisiert dessen Stack.
- `void OS_KernelStart (void)`
Startet den RTOS-Kernel und übergibt die Kontrolle an den ersten Task.
- `void OS_Scheduler (void)`
Führt die Task-Planung (Scheduling) durch.
- `__attribute__((naked))`
Startet den ersten Task (Assembler-Einsprung).
- `void OS_Delay (uint32_t tick_time)`
Blockiert den aktuellen Task für eine definierte Anzahl an Ticks.

Variables

- `TCB_sctTCB_t g_asctTCBs [Max_Task]`
Array zur Speicherung der Task Control Blocks (TCBs).
- `TCB_sctTCB_t * g_pCurrentTCB`
Zeiger auf den TCB des aktuell auszuführenden Tasks.
- `uint32_t MAX_TASKS`
Externe Variable, definiert die absolute maximale Task-Anzahl.

4.4.1 Detailed Description

Kernfunktionen des Real-Time Operating Systems (RTOS).

Diese Datei enthält die Kernverwaltung des RTOS, einschließlich der Task-Erstellung, der Initialisierung des Schedulers, der Kontextwechsel-Logik sowie der Delay-Funktionalität. Der Scheduler implementiert ein präemptives Round-Robin (RR) Verfahren unter Berücksichtigung eines Idle-Tasks.

Author

Ezechiel Tonkeme

Date

10. Mai 2026

4.4.2 Macro Definition Documentation

4.4.2.1 Max_Task

```
#define Max_Task 3
```

Maximale Anzahl an Tasks, die vom OS verwaltet werden (inkl. Idle-Task).

4.4.3 Typedef Documentation

4.4.3.1 OS_TaskFunction_t

```
typedef void(* OS_TaskFunction_t) (void)
```

4.4.4 Function Documentation

4.4.4.1 __attribute__((naked))

```
__attribute__((naked))
```

Startet den ersten Task (Assembler-Einsprung).

•

Das Attribut `__attribute__((naked))` verhindert, dass der C-Compiler eigenen Stack-Code hinzufügt, der unsere Register durcheinanderbringt. Die Funktion lädt den Stackpointer des ersten Tasks, stellt die Register wieder her und führt einen Branch (`bx r3`) zur Startadresse aus.

4.4.4.2 OS_Delay()

```
void OS_Delay (
    uint32_t tick_time)
```

Blockiert den aktuellen Task für eine definierte Anzahl an Ticks.

Setzt den Task in den Zustand `TaskState_Blocked`, weist ihm die übergebene Wartezeit zu und erzwingt sofort einen Context Switch durch Auslösen des `PendSV`-Interrupts. Dadurch wird die CPU an den nächsten bereiten Task abgegeben. Interrupts werden kurzzeitig deaktiviert, um Dateninkonsistenzen zu vermeiden.

Parameters

<i>tick_time</i>	Anzahl der OS-Ticks (Millisekunden), für die der Task pausieren soll.
------------------	---

4.4.4.3 OS_KernelStart()

```
void OS_KernelStart (
    void )
```

Startet den RTOS-Kernel und übergibt die Kontrolle an den ersten Task.

Sucht den Task mit der höchsten Priorität (niedrigster Zahlenwert), setzt die `PendSV`-Priorität auf den niedrigsten Wert (um Interrupt-Konflikte zu vermeiden), konfiguriert den `SysTick`-Timer für eine Millisekunde und führt den Assembler-Sprung in den ersten Task aus.

4.4.4.4 OS_Scheduler()

```
void OS_Scheduler (
    void )
```

Führt die Task-Planung (Scheduling) durch.

Implementiert ein Round-Robin-Scheduling. Es wird im Array der TCBs zyklisch nach dem nächsten Task im Zustand `TaskState_Ready` gesucht. Der Idle-Task (letzter Task im Array) wird dabei zunächst übersprungen. Findet sich kein Ready-Task, wird der Idle-Task in den Zustand `TaskState_Running` versetzt.

4.4.4.5 OS_StartFirstTask()

```
void OS_StartFirstTask (
    void )
```

4.4.4.6 OS_TaskCreate()

```
uint8_t OS_TaskCreate (
    OS_TaskFunction_t pTaskCode,
    uint8_t u8Prio,
    uint8_t u8TaskId)
```

Erstellt einen neuen Task und initialisiert dessen Stack.

Diese Funktion weist dem neuen Task einen TCB zu und bereitet den Stack-Frame so vor, als wäre der Task durch einen Interrupt unterbrochen worden. Dies umfasst den Hardware-Stack (xPSR, PC, LR, R12, R3-R0) sowie den Software-Stack (R11-R4) für die ARM Cortex-M Architektur.

Parameters

<i>pTaskCode</i>	Funktionszeiger auf die auszuführende Task-Funktion.
<i>u8Prio</i>	Priorität des Tasks (niedriger Wert = höhere Priorität).
<i>u8TaskId</i>	Eindeutige ID des Tasks.

Returns

0 bei erfolgreicher Erstellung, 1 wenn die maximale Task-Anzahl (Max_Task) erreicht ist.

4.4.5 Variable Documentation

4.4.5.1 g_asctTCBs

```
TCB_sctTCB_t g_asctTCBs[Max_Task]
```

Array zur Speicherung der Task Control Blocks (TCBs).

4.4.5.2 g_pCurrentTCB

```
TCB_sctTCB_t* g_pCurrentTCB
```

Zeiger auf den TCB des aktuell auszuführenden Tasks.

4.4.5.3 MAX_TASKS

```
uint32_t MAX_TASKS [extern]
```

Externe Variable, definiert die absolute maximale Task-Anzahl.

4.5 stm32l4xx_hal_msp.c File Reference

This file provides code for the MSP Initialization and de-Initialization codes.

```
#include "main.h"
```

Functions

- void [HAL_MspInit](#) (void)
- void [HAL_DFSDM_ChannelMspInit](#) (DFSDM_Channel_HandleTypeDef *hdfsdm_channel)
DFSDM_Channel MSP Initialization This function configures the hardware resources used in this example.
- void [HAL_DFSDM_ChannelMspDeInit](#) (DFSDM_Channel_HandleTypeDef *hdfsdm_channel)
DFSDM_Channel MSP De-Initialization This function freeze the hardware resources used in this example.
- void [HAL_I2C_MspInit](#) (I2C_HandleTypeDef *hi2c)
I2C MSP Initialization This function configures the hardware resources used in this example.
- void [HAL_I2C_MspDeInit](#) (I2C_HandleTypeDef *hi2c)
I2C MSP De-Initialization This function freeze the hardware resources used in this example.
- void [HAL_QSPI_MspInit](#) (QSPI_HandleTypeDef *hqspi)
QSPI MSP Initialization This function configures the hardware resources used in this example.
- void [HAL_QSPI_MspDeInit](#) (QSPI_HandleTypeDef *hqspi)
QSPI MSP De-Initialization This function freeze the hardware resources used in this example.
- void [HAL_SPI_MspInit](#) (SPI_HandleTypeDef *hspi)
SPI MSP Initialization This function configures the hardware resources used in this example.
- void [HAL_SPI_MspDeInit](#) (SPI_HandleTypeDef *hspi)
SPI MSP De-Initialization This function freeze the hardware resources used in this example.
- void [HAL_UART_MspInit](#) (UART_HandleTypeDef *huart)
UART MSP Initialization This function configures the hardware resources used in this example.
- void [HAL_UART_MspDeInit](#) (UART_HandleTypeDef *huart)
UART MSP De-Initialization This function freeze the hardware resources used in this example.
- void [HAL_PCD_MspInit](#) (PCD_HandleTypeDef *hpcd)
PCD MSP Initialization This function configures the hardware resources used in this example.
- void [HAL_PCD_MspDeInit](#) (PCD_HandleTypeDef *hpcd)
PCD MSP De-Initialization This function freeze the hardware resources used in this example.

4.5.1 Detailed Description

This file provides code for the MSP Initialization and de-Initialization codes.

Attention

Copyright (c) 2026 STMicroelectronics. All rights reserved.

This software is licensed under terms that can be found in the LICENSE file in the root directory of this software component. If no LICENSE file comes with this software, it is provided AS-IS.

4.5.2 Function Documentation

4.5.2.1 HAL_DFSDM_ChannelMspDeInit()

```
void HAL_DFSDM_ChannelMspDeInit (
    DFSDM_Channel_HandleTypeDef * hdfsdm_channel)
```

DFSDM_Channel MSP De-Initialization This function freeze the hardware resources used in this example.

Parameters

<i>hdfsdm_channel</i>	DFSDM_Channel handle pointer
-----------------------	------------------------------

Return values

<i>None</i>	
-------------	--

DFSDM1 GPIO Configuration PE7 -----> DFSDM1_DATIN2 PE9 -----> DFSDM1_CKOUT

4.5.2.2 HAL_DFSDM_ChannelMspInit()

```
void HAL_DFSDM_ChannelMspInit (
    DFSDM_Channel_HandleTypeDef * hdfsdm_channel)
```

DFSDM_Channel MSP Initialization This function configures the hardware resources used in this example.

Parameters

<i>hdfsdm_channel</i>	DFSDM_Channel handle pointer
-----------------------	------------------------------

Return values

<i>None</i>	
-------------	--

Initializes the peripherals clock

DFSDM1 GPIO Configuration PE7 -----> DFSDM1_DATIN2 PE9 -----> DFSDM1_CKOUT

4.5.2.3 HAL_I2C_MspDeInit()

```
void HAL_I2C_MspDeInit (
    I2C_HandleTypeDef * hi2c)
```

I2C MSP De-Initialization This function freeze the hardware resources used in this example.

Parameters

<i>hi2c</i>	I2C handle pointer
-------------	--------------------

Return values

<i>None</i>	
-------------	--

I2C2 GPIO Configuration PB10 -----> I2C2_SCL PB11 -----> I2C2_SDA

4.5.2.4 HAL_I2C_MspInit()

```
void HAL_I2C_MspInit (
    I2C_HandleTypeDef * hi2c)
```

I2C MSP Initialization This function configures the hardware resources used in this example.

Parameters

<i>hi2c</i>	I2C handle pointer
-------------	--------------------

Return values

<i>None</i>	
-------------	--

Initializes the peripherals clock

I2C2 GPIO Configuration PB10 -----> I2C2_SCL PB11 -----> I2C2_SDA

4.5.2.5 HAL_MspInit()

```
void HAL_MspInit (
    void )
```

Initializes the Global MSP.

4.5.2.6 HAL_PCD_MspDeInit()

```
void HAL_PCD_MspDeInit (
    PCD_HandleTypeDef * hpcd)
```

PCD MSP De-Initialization This function freeze the hardware resources used in this example.

Parameters

<i>hpcd</i>	PCD handle pointer
-------------	--------------------

Return values

<i>None</i>	
-------------	--

USB_OTG_FS GPIO Configuration PA9 -----> USB_OTG_FS_VBUS PA10 -----> USB_OTG_FS_ID PA11 ----->
USB_OTG_FS_DM PA12 -----> USB_OTG_FS_DP

4.5.2.7 HAL_PCD_MspInit()

```
void HAL_PCD_MspInit (
    PCD_HandleTypeDef * hpcd)
```

PCD MSP Initialization This function configures the hardware resources used in this example.

Parameters

<i>hpcd</i>	PCD handle pointer
-------------	--------------------

Return values

<i>None</i>	
-------------	--

Initializes the peripherals clock

USB_OTG_FS GPIO Configuration PA9 -----> USB_OTG_FS_VBUS PA10 -----> USB_OTG_FS_ID PA11 ----->
USB_OTG_FS_DM PA12 -----> USB_OTG_FS_DP

4.5.2.8 HAL_QSPI_MspDeInit()

```
void HAL_QSPI_MspDeInit (
    QSPI_HandleTypeDef * hqspi)
```

QSPI MSP De-Initialization This function freeze the hardware resources used in this example.

Parameters

<i>hqspi</i>	QSPI handle pointer
--------------	---------------------

Return values

<i>None</i>	
-------------	--

QUADSPI GPIO Configuration PE10 -----> QUADSPI_CLK PE11 -----> QUADSPI_NCS PE12 ----->
QUADSPI_BK1_IO0 PE13 -----> QUADSPI_BK1_IO1 PE14 -----> QUADSPI_BK1_IO2 PE15 -----> QUADSPI_BK1_IO3

4.5.2.9 HAL_QSPI_MspInit()

```
void HAL_QSPI_MspInit (
    QSPI_HandleTypeDef * hqspi)
```

QSPI MSP Initialization This function configures the hardware resources used in this example.

Parameters

<i>hqspi</i>	QSPI handle pointer
--------------	---------------------

Return values

<i>None</i>	
-------------	--

QUADSPI GPIO Configuration PE10 -----> QUADSPI_CLK PE11 -----> QUADSPI_NCS PE12 -----> QUADSPI_BK1_IO0 PE13 -----> QUADSPI_BK1_IO1 PE14 -----> QUADSPI_BK1_IO2 PE15 -----> QUADSPI_BK1_IO3

4.5.2.10 HAL_SPI_MspDeInit()

```
void HAL_SPI_MspDeInit (
    SPI_HandleTypeDef * hspi)
```

SPI MSP De-Initialization This function freeze the hardware resources used in this example.

Parameters

<i>hspi</i>	SPI handle pointer
-------------	--------------------

Return values

<i>None</i>	
-------------	--

SPI3 GPIO Configuration PC10 -----> SPI3_SCK PC11 -----> SPI3_MISO PC12 -----> SPI3_MOSI

4.5.2.11 HAL_SPI_MspInit()

```
void HAL_SPI_MspInit (
    SPI_HandleTypeDef * hspi)
```

SPI MSP Initialization This function configures the hardware resources used in this example.

Parameters

<i>hspi</i>	SPI handle pointer
-------------	--------------------

Return values

<i>None</i>	
-------------	--

SPI3 GPIO Configuration PC10 -----> SPI3_SCK PC11 -----> SPI3_MISO PC12 -----> SPI3_MOSI

4.5.2.12 HAL_UART_MspDeInit()

```
void HAL_UART_MspDeInit (  
    UART_HandleTypeDef * huart)
```

UART MSP De-Initialization This function freeze the hardware resources used in this example.

Parameters

<i>huart</i>	UART handle pointer
--------------	---------------------

Return values

<i>None</i>	
-------------	--

USART1 GPIO Configuration PB6 -----> USART1_TX PB7 -----> USART1_RX

USART3 GPIO Configuration PD8 -----> USART3_TX PD9 -----> USART3_RX

4.5.2.13 HAL_UART_MspInit()

```
void HAL_UART_MspInit (  
    UART_HandleTypeDef * huart)
```

UART MSP Initialization This function configures the hardware resources used in this example.

Parameters

<i>huart</i>	UART handle pointer
--------------	---------------------

Return values

<i>None</i>	
-------------	--

Initializes the peripherals clock

USART1 GPIO Configuration PB6 -----> USART1_TX PB7 -----> USART1_RX

Initializes the peripherals clock

USART3 GPIO Configuration PD8 -----> USART3_TX PD9 -----> USART3_RX

4.6 stm32l4xx_it.c File Reference

Interrupt Service Routines.

```
#include "main.h"
#include "stm32l4xx_it.h"
#include "os_kernel.h"
#include "tcb.h"
```

Macros

- #define [TIME_SLICE_TICKS](#) 5
- #define [Max_Task](#) 3

Functions

- void [OS_Scheduler](#) (void)
Führt die Task-Planung (Scheduling) durch.
- void [NMI_Handler](#) (void)
This function handles Non maskable interrupt.
- void [HardFault_Handler](#) (void)
This function handles Hard fault interrupt.
- void [MemManage_Handler](#) (void)
This function handles Memory management fault.
- void [BusFault_Handler](#) (void)
This function handles Prefetch fault, memory access fault.
- void [UsageFault_Handler](#) (void)
This function handles Undefined instruction or illegal state.
- void [SVC_Handler](#) (void)
This function handles System service call via SWI instruction.
- void [DebugMon_Handler](#) (void)
This function handles Debug monitor.
- [__attribute__](#) ((naked))
This function handles Pendable request for system service.
- void [SysTick_Handler](#) (void)
Behandelt den System Tick Timer Interrupt (SysTick).
- void [EXTI9_5_IRQHandler](#) (void)
This function handles EXTI line[9:5] interrupts.
- void [EXTI15_10_IRQHandler](#) (void)
This function handles EXTI line[15:10] interrupts.

Variables

- TCB_sctTCB_t * [g_pCurrentTCB](#)
Zeiger auf den TCB des aktuell auszuführenden Tasks.
- TCB_sctTCB_t [g_asctTCBs](#) []
Array zur Speicherung der Task Control Blocks (TCBs).

4.6.1 Detailed Description

Interrupt Service Routines.

Attention

Copyright (c) 2026 STMicroelectronics. All rights reserved.

This software is licensed under terms that can be found in the LICENSE file in the root directory of this software component. If no LICENSE file comes with this software, it is provided AS-IS.

4.6.2 Macro Definition Documentation

4.6.2.1 Max_Task

```
#define Max_Task 3
```

4.6.2.2 TIME_SLICE_TICKS

```
#define TIME_SLICE_TICKS 5
```

4.6.3 Function Documentation

4.6.3.1 __attribute__((naked))

```
__attribute__((naked))
```

This function handles Pendable request for system service.

Führt den Context Switch (Kontextwechsel) durch.

Diese Funktion ist der PendSV-Interrupt-Handler. Das Attribut `__attribute__((naked))` verhindert, dass der Compiler eigenen Prolog/Epilog-Code generiert. Die Funktion sichert die verbleibenden Software-Register (r4-r11) des aktuell laufenden Tasks auf dessen Stack, speichert den aktuellen Stackpointer (MSP) im Task Control Block (TCB) und ruft den `OS_Scheduler` auf, um den nächsten Task zu bestimmen. Danach lädt sie den Stackpointer des neuen Tasks, stellt dessen Software-Register wieder her und triggert über den Rücksprung (`bx lr` mit `EXC_RETURN`) die automatische Wiederherstellung der Hardware-Register durch die CPU. Der Context Switch.

4.6.3.2 BusFault_Handler()

```
void BusFault_Handler (void)
```

This function handles Prefetch fault, memory access fault.

4.6.3.3 DebugMon_Handler()

```
void DebugMon_Handler (
    void )
```

This function handles Debug monitor.

4.6.3.4 EXTI15_10_IRQHandler()

```
void EXTI15_10_IRQHandler (
    void )
```

This function handles EXTI line[15:10] interrupts.

4.6.3.5 EXTI9_5_IRQHandler()

```
void EXTI9_5_IRQHandler (
    void )
```

This function handles EXTI line[9:5] interrupts.

4.6.3.6 HardFault_Handler()

```
void HardFault_Handler (
    void )
```

This function handles Hard fault interrupt.

4.6.3.7 MemManage_Handler()

```
void MemManage_Handler (
    void )
```

This function handles Memory management fault.

4.6.3.8 NMI_Handler()

```
void NMI_Handler (
    void )
```

This function handles Non maskable interrupt.

4.6.3.9 OS_Scheduler()

```
void OS_Scheduler (
    void ) [extern]
```

Führt die Task-Planung (Scheduling) durch.

Implementiert ein Round-Robin-Scheduling. Es wird im Array der TCBs zyklisch nach dem nächsten Task im Zustand `TaskState_Ready` gesucht. Der Idle-Task (letzter Task im Array) wird dabei zunächst übersprungen. Findet sich kein Ready-Task, wird der Idle-Task in den Zustand `TaskState_Running` versetzt.

4.6.3.10 SVC_Handler()

```
void SVC_Handler (
    void )
```

This function handles System service call via SWI instruction.

4.6.3.11 SysTick_Handler()

```
void SysTick_Handler (
    void )
```

Behandelt den System Tick Timer Interrupt (SysTick).

Diese Funktion wird periodisch von der Hardware aufgerufen (z.B. jede Millisekunde). Sie übernimmt neben der grundlegenden HAL-Zeitbasis (HAL_IncTick) drei zentrale Aufgaben für das RTOS:

1. Sie verringert die Timeout-Zähler (u32TimeoutTicks) von blockierten Tasks (z. B. Wartezeiten bei Semaphoren/Queues) und weckt diese bei Ablauf auf.
2. Sie aktualisiert die Delay-Zähler (u32DelayTicks) von pausierten Tasks (ausgelöst durch [OS_Delay](#)) und setzt diese bei Ablauf wieder auf TaskState_Ready.
3. Sie verwaltet das Time-Slicing für das präemptive Round-Robin-Scheduling. Sobald die festgelegte Zeitscheibe (g_u8PraemtiveTime) abgelaufen ist, wird ein PendSV-Interrupt ausgelöst, um den Context Switch zu erzwingen.

4.6.3.12 UsageFault_Handler()

```
void UsageFault_Handler (
    void )
```

This function handles Undefined instruction or illegal state.

4.6.4 Variable Documentation

4.6.4.1 g_asctTCBs

```
TCB_sctTCB_t g_asctTCBs[] [extern]
```

Array zur Speicherung der Task Control Blocks (TCBs).

4.6.4.2 g_pCurrentTCB

```
TCB_sctTCB_t* g_pCurrentTCB [extern]
```

Zeiger auf den TCB des aktuell auszuführenden Tasks.

4.7 syscalls.c File Reference

STM32CubeIDE Minimal System calls file.

```
#include <sys/stat.h>
#include <stdlib.h>
#include <errno.h>
#include <stdio.h>
#include <signal.h>
#include <time.h>
#include <sys/time.h>
#include <sys/times.h>
```

Functions

- int `__io_putchar` (int ch) `__attribute__((weak))`
- int `__io_getchar` (void)
- void `initialise_monitor_handles` ()
- int `_getpid` (void)
- int `_kill` (int pid, int sig)
- void `_exit` (int status)
- `__attribute__((weak))`
- int `_close` (int file)
- int `_fstat` (int file, struct stat *st)
- int `_isatty` (int file)
- int `_lseek` (int file, int ptr, int dir)
- int `_open` (char *path, int flags,...)
- int `_wait` (int *status)
- int `_unlink` (char *name)
- int `_times` (struct tms *buf)
- int `_stat` (char *file, struct stat *st)
- int `_link` (char *old, char *new)
- int `_fork` (void)
- int `_execve` (char *name, char **argv, char **env)

Variables

- char ** `environ` = `__env`

4.7.1 Detailed Description

STM32CubeIDE Minimal System calls file.

Author

Auto-generated by STM32CubeIDE

For more information about which c-functions
need which of these lowlevel functions
please consult the Newlib libc-manual

Attention

Copyright (c) 2020-2025 STMicroelectronics. All rights reserved.

This software is licensed under terms that can be found in the LICENSE file in the root directory of this software component. If no LICENSE file comes with this software, it is provided AS-IS.

4.7.2 Function Documentation

4.7.2.1 `__attribute__()`

```
__attribute__ (  
    (weak) )
```

4.7.2.2 `__io_getchar()`

```
int __io_getchar (  
    void ) [extern]
```

4.7.2.3 `__io_putchar()`

```
int __io_putchar (  
    int ch) [extern]
```

4.7.2.4 `_close()`

```
int _close (  
    int file)
```

4.7.2.5 `_execve()`

```
int _execve (  
    char * name,  
    char ** argv,  
    char ** env)
```

4.7.2.6 `_exit()`

```
void _exit (  
    int status)
```

4.7.2.7 `_fork()`

```
int _fork (  
    void )
```

4.7.2.8 `_fstat()`

```
int _fstat (  
    int file,  
    struct stat * st)
```

4.7.2.9 `_getpid()`

```
int _getpid (  
    void )
```

4.7.2.10 `_isatty()`

```
int _isatty (  
    int file)
```

4.7.2.11 `_kill()`

```
int _kill (  
    int pid,  
    int sig)
```

4.7.2.12 `_link()`

```
int _link (  
    char * old,  
    char * new)
```

4.7.2.13 `_lseek()`

```
int _lseek (  
    int file,  
    int ptr,  
    int dir)
```

4.7.2.14 `_open()`

```
int _open (  
    char * path,  
    int flags,  
    ...)
```

4.7.2.15 `_stat()`

```
int _stat (  
    char * file,  
    struct stat * st)
```

4.7.2.16 `_times()`

```
int _times (  
    struct tms * buf)
```


4.7.2.17 _unlink()

```
int _unlink (  
    char * name)
```

4.7.2.18 _wait()

```
int _wait (  
    int * status)
```

4.7.2.19 initialise_monitor_handles()

```
void initialise_monitor_handles ()
```

4.7.3 Variable Documentation

4.7.3.1 environ

```
char** environ = __env
```

4.8 sysmem.c File Reference

STM32CubeIDE System Memory calls file.

```
#include <errno.h>  
#include <stdint.h>
```

Functions

- void * [_sbrk](#) (ptrdiff_t incr)
[_sbrk\(\)](#) allocates memory to the newlib heap and is used by malloc and others from the C library

4.8.1 Detailed Description

STM32CubeIDE System Memory calls file.

Author

Generated by STM32CubeIDE

```
For more information about which C functions  
need which of these lowlevel functions  
please consult the newlib libc manual
```

Attention

Copyright (c) 2025 STMicroelectronics. All rights reserved.

This software is licensed under terms that can be found in the LICENSE file in the root directory of this software component. If no LICENSE file comes with this software, it is provided AS-IS.

4.8.2 Function Documentation

4.8.2.1 `_sbrk()`

```
void * _sbrk (
    ptrdiff_t incr)
```

`_sbrk()` allocates memory to the newlib heap and is used by malloc and others from the C library

```
* #####
* # .data # .bss #          newlib heap          #          MSP stack          #
* #          #          #          #          # Reserved by _Min_Stack_Size #
* #####
* ^-- RAM start          ^-- _end          _estack, RAM end --^
*
```

This implementation starts allocating at the '`_end`' linker symbol The '`_Min_Stack_Size`' linker symbol reserves a memory for the MSP stack The implementation considers '`_estack`' linker symbol to be RAM end NOTE: If the MSP stack, at any point during execution, grows larger than the reserved size, please increase the '`_Min_Stack_Size`'.

Parameters

<code>incr</code>	Memory size
-------------------	-------------

Returns

Pointer to allocated memory

4.9 `system_stm32l4xx.c` File Reference

CMSIS Cortex-M4 Device Peripheral Access Layer System Source File.

```
#include "stm32l4xx.h"
```

Macros

- `#define HSE_VALUE 8000000U`
- `#define MSI_VALUE 4000000U`
- `#define HSI_VALUE 16000000U`

Functions

- void `SystemInit` (void)
Setup the microcontroller system.
- void `SystemCoreClockUpdate` (void)
Update SystemCoreClock variable according to Clock Register Values. The SystemCoreClock variable contains the core clock (HCLK), it can be used by the user application to setup the SysTick timer or configure other parameters.

Variables

- uint32_t [SystemCoreClock](#) = 4000000U
- const uint8_t [AHBPrescTable](#) [16] = {0U, 0U, 0U, 0U, 0U, 0U, 0U, 0U, 1U, 2U, 3U, 4U, 6U, 7U, 8U, 9U}
- const uint8_t [APBPrescTable](#) [8] = {0U, 0U, 0U, 0U, 1U, 2U, 3U, 4U}
- const uint32_t [MSIRangeTable](#) [12]

4.9.1 Detailed Description

CMSIS Cortex-M4 Device Peripheral Access Layer System Source File.

Author

MCD Application Team

This file provides two functions and one global variable to be called from user application:

- [SystemInit\(\)](#): This function is called at startup just after reset and before branch to main program. This call is made inside the "startup_stm32l4xx.s" file.
- SystemCoreClock variable: Contains the core clock (HCLK), it can be used by the user application to setup the SysTick timer or configure other parameters.
- [SystemCoreClockUpdate\(\)](#): Updates the variable SystemCoreClock and must be called whenever the core clock is changed during program execution.

After each device reset the MSI (4 MHz) is used as system clock source. Then [SystemInit\(\)](#) function is called, in "startup_stm32l4xx.s" file, to configure the system clock before to branch to main program.

4.9.2 This file configures the system clock as follows:

4.9.2.1 System Clock source | MSI

4.9.2.2 SYSCLK(Hz) | 4000000

4.9.2.3 HCLK(Hz) | 4000000

4.9.2.4 AHB Prescaler | 1

4.9.2.5 APB1 Prescaler | 1

4.9.2.6 APB2 Prescaler | 1

4.9.2.7 PLL_M | 1

4.9.2.8 PLL_N | 8

4.9.2.9 PLL_P | 7

4.9.2.10 PLL_Q | 2

4.9.2.11 PLL_R | 2

4.9.2.12 PLLSAI1_P | NA

4.9.2.13 PLLSAI1_Q | NA

4.9.2.14 PLLSAI1_R | NA

4.9.2.15 PLLSAI2_P | NA

4.9.2.16 PLLSAI2_Q | NA

4.9.2.17 PLLSAI2_R | NA

Require 48MHz for USB OTG FS, | Disabled

4.9.2.18 SDIO and RNG clock |

=====

Attention

Copyright (c) 2017 STMicroelectronics. All rights reserved.

This software is licensed under terms that can be found in the LICENSE file in the root directory of this software component. If no LICENSE file comes with this software, it is provided AS-IS.